

Руководство пользователя

Документ описывает, как установить проект, запустить основные сценарии, какие файлы получаются на выходе и как интерпретировать результат.

Установка

Перейти в папку проекта:

```
cd "G:\BIG DZ ARCHIEVE\archive dz 8Sem\Диплом\positioning"
```

Установить пакет в editable-режиме с тестовыми зависимостями и экспортом документации:

```
python -m pip install -e ".[dev,docs]"
```

Проверить установку:

```
positioning demo --config .\configs\default.json  
python -m pytest
```

Конфигурация

По умолчанию используется configs/default.json. В нем задаются:

- координаты четырех микрофонов;
- частота дискретизации;
- размер окна;
- перекрытие окон;
- частотная полоса;
- температура;
- влажность;
- режим локализации.

Чтобы проверить влияние температуры, достаточно изменить environment.temperature_c и снова запустить positioning demo. Скорость звука будет пересчитана автоматически.

Чтобы переключить дальнепольный режим, изменить:

```
"localization": {  
  "mode": "direction"  
}
```

Демонстрационный запуск

```
positioning demo --config .\configs\default.json
```

Команда выводит:

- статус;
- режим;
- надежность;
- доверие;
- координаты или направление;
- азимут и угол места;
- невязку TDOA;
- число использованных и отброшенных пар;

- скорость звука.

Сохранить результат в JSON:

```
positioning demo --json-output .\data\logs\demo_result.json
```

Синтетический 4-канальный сигнал

Создать CSV-сигнал:

```
positioning generate-synthetic --output .\data\synthetic\signal.csv
```

Для демонстрации GCC-PHAT лучше использовать широкополосный импульс:

```
positioning generate-synthetic --output .\data\synthetic\signal.csv --source-kind pulse
```

Создать WAV-сигнал:

```
positioning generate-synthetic --format wav --output .\data\synthetic\signal.wav --source-kind pulse
```

Проверить прием WAV и метаданных для пункта 2 ПМИ:

```
positioning inspect-audio --input .\data\synthetic\signal.wav --require-metadata
```

Ожидаемый вывод содержит:

- Статус: ok;
- Формат: WAV;
- Каналов: 4;
- Частота дискретизации: 48000 Гц;
- Метаданные: найдены;
- координаты цели и скорость звука из sidecar-файла.

Пример с шумом, выбросом и отражением:

```
positioning generate-synthetic `
  --output .\data\synthetic\hard_case.csv `
  --target-x 0.6 `
  --target-y 0.4 `
  --target-z 0.3 `
  --source-kind pulse `
  --noise-std 0.02 `
  --outlier-channel 2 `
  --outlier-time 0.15 `
  --outlier-amplitude 0.8 `
  --reflection-delay 0.002 `
  --reflection-gain 0.25
```

Рядом с сигналом создается JSON-файл метаданных. Например для signal.csv будет создан signal.meta.json.

Синтетическая траектория

Для визуальной демонстрации потоковой обработки нужен не один неподвижный источник, а движущаяся цель. Такой файл создается командой:

```
positioning generate-trajectory `
  --output .\data\synthetic\trajectory_demo.csv `
  --duration 1.2 `
  --point-count 12 `
  --noise-std 0.001
```

В результате создаются:

- trajectory_demo.csv - четырехканальный сигнал с серией импульсов;
- trajectory_demo.csv.meta.json - metadata с массивом trajectory.

Каждый элемент trajectory содержит:

- timestamp - время излучения импульса;
- x, y, z - координаты источника в этот момент.

По умолчанию траектория идет от точки (0.35, 0.25, 0.3) м к точке (0.9, 0.55, 0.3) м.

Координаты можно изменить параметрами --start-x, --start-y, --start-z, --end-x, --end-y, --end-z.

Формат CSV-сигнала

CSV содержит четыре числовых столбца: по одному каналу на микрофон. Частота дискретизации берется из конфигурации. Если каналов меньше или больше четырех, чтение завершается понятной ошибкой.

Формат WAV-сигнала

WAV должен быть четырехканальным. Данные приводятся к внутреннему представлению AudioFrame. Этот режим нужен для офлайн-проверок и будущей работы с реальными записями.

Предобработка

Запуск:

```
positioning preprocess --input .\data\synthetic\signal.csv --output-dir .\data\processed
```

Для WAV:

```
positioning preprocess --input .\data\synthetic\signal.wav --output-dir .\data\processed_wav
```

На выходе создаются:

- window_0000.csv, window_0001.csv и так далее;
- preprocess.meta.json с параметрами обработки и диагностикой каждого окна.

Окно считается информативным, если оно имеет достаточную энергию и приемлемую межканальную согласованность.

Визуализация потоковой обработки

Для демонстрации пункта 3 ПМИ сначала выполняется чистая предобработка:

```
positioning preprocess `
  --input .\data\synthetic\trajectory_demo.csv `
  --output-dir .\data\processed_demo
```

Затем команда локализации может сразу открыть всплывающее окно с видом сверху:

```
positioning localization `
  --input .\data\synthetic\trajectory_demo.csv `
  --mode position `
  --show-visualization `
  --trajectory-max-windows 45
```

В окне отображаются:

- синие точки - микрофоны решетки;
- зеленая точка или линия - исходная траектория из .meta.json;
- красные точки или линия - распознанные оценки по окнам;
- пунктирные линии - расхождение исходной и распознанной точек.

Если в metadata есть массив trajectory, окно отрисует всю исходную траекторию зеленой линией. Если metadata содержит только одну цель target, программа выводит предупреждение и ограничивает визуализацию первым окном, чтобы не показывать шумовые выбросы как траекторию.

Для одиночной точки используется тот же механизм:

```
positioning localization `
--input .\data\synthetic\signal.csv `
--mode direction `
--max-windows 1 `
--show-visualization
```

Полезные параметры:

- --mode position - считать и рисовать распознанные координаты;
- --mode direction - считать направление и рисовать его как точку на заданной дальности;
- --trajectory-max-windows - ограничить число окон для быстрой демонстрации;
- --trajectory-min-confidence - не показывать низкодостоверные оценки;
- --trajectory-max-error - отбрасывать точки, которые слишком далеко ушли от исходной траектории;
- --trajectory-direction-range - дальность отрисовки для режима direction.

Локализация из аудиофайла через GCC-PHAT

Запуск полного файлового конвейера:

```
positioning localize-audio `
--input .\data\synthetic\signal.csv `
--mode direction `
--max-windows 1 `
--csv-log .\data\logs\audio_localization.csv `
--json-output .\data\logs\audio_final_result.json
```

Для WAV используется та же команда:

```
positioning localize-audio --input .\data\synthetic\signal.wav --mode direction
```

Что делает команда:

1. читает 4-канальный CSV или WAV;
2. разбивает сигнал на окна из JSON-конфигурации;
3. выполняет предобработку каждого окна для диагностики информативности;
4. оценивает задержки для всех шести пар микрофонов через GCC-PHAT с внутренним удалением DC и RMS-нормализацией каналов;
5. отбраковывает физически невозможные и низкокачественные пары;
6. считает направление или координаты;
7. сглаживает поток результатов;
8. сохраняет CSV-журнал и итоговый JSON.

Полезные параметры:

- --mode direction - дальнепольный режим, основной для демонстрации БПЛА;
- --mode position - ближнепольный/расширенный 3D-режим;
- --max-windows 5 - обработать только первые пять окон;
- --interpolation 8 - интерполяция GCC-PHAT для дробной оценки задержки;
- --min-quality 0.2 - минимальное качество TDOA-пары.

CSV-журнал содержит TDOA-пары в поле `tdoa_pairs_json`. Для каждой пары сохраняются задержка, качество и вес.

Валидация решателя

```
positioning validate --config .\configs\default.json
```

Эта команда запускает синтетические сценарии для проверки TDOA-локализации. Она полезна после изменения геометрии решетки, температуры, режима локализации или кода решателя.

Сопровождение цели

```
positioning track-demo `
--csv-log .\data\logs\tracking.csv `
--json-output .\data\logs\final_result.json `
--inject-low-confidence-jump
```

Команда имитирует поток окон. При включенном `--inject-low-confidence-jump` в середину последовательности добавляется резкий низкодостоверный скачок. Трекер должен отбраковать его или снизить влияние на сглаженную оценку.

CSV-журнал содержит:

- индекс окна;
- timestamp;
- статус;
- режим;
- азимут;
- угол места;
- доверие;
- надежность;
- невязку;
- число использованных/отброшенных пар;
- TDOA-пары;
- параметры среды и окна.

ПМИ

Запуск полного набора проверок:

```
positioning pmi --output-dir .\data\pmi --repeat-count 100
```

Результаты:

- data/pmi/ПРОТОКОЛ_ПМИ.md - человекочитаемый протокол;
- data/pmi/pmi_results.json - машинный JSON;
- data/pmi/pmi_results.csv - таблица для отчета.

Проверки включают запуск программы, прием четырех каналов, файловый режим, учет температуры и влажности, фильтрацию шумного сигнала, корректность TDOA, подавление отраженного пути, расчет направления/координат, JSON/CSV-выход, время обработки окна и нештатные ситуации.

Тесты

```
python -m pytest
```

Тесты покрывают:

- геометрию и модели данных;

- расчет скорости звука;
- загрузку корректного и некорректного JSON;
- синтетическую генерацию;
- CSV/WAV вход-выход;
- предобработку;
- локализацию и робастность;
- сопровождение и вывод;
- CLI smoke-тесты;
- ПМИ.

Экспорт документации в PDF

Сгенерировать PDF-файлы:

```
python .\tools\export_docs_to_pdf.py
```

Результаты сохраняются в docs/pdf:

- отдельный PDF для каждого Markdown-документа;
- общий файл `positioning_documentation.pdf`, объединяющий README и основные документы из docs.

Экспорт использует reportlab и системные шрифты Windows с поддержкой кириллицы.

Частые ошибки

Ошибка конфигурации означает, что JSON не прошел валидацию. Проверить нужно число микрофонов, координаты, `sample_rate`, `window_size`, `overlap`, частотную полосу, температуру, влажность и режим.

Ошибка чтения сигнала обычно означает, что файл не найден, CSV/WAV имеет не четыре канала или данные не являются числовыми.

Статус `insufficient_signal` означает, что окно слишком тихое или каналы плохо согласованы. Для синтетики можно увеличить амплитуду или снизить шум, для реального сигнала - проверить запись и настройки полосы.

Статус `low_confidence` означает, что решение найдено, но невязка, качество пар или согласованность недостаточны для уверенного результата.